pyflightstream User Guide

A version-aware, didactic Python driver for the FlightStream solver

Geovana Neves

pyflightstream 0.39.0, MIT licensed

October 2026

Guide roadmap

Part I Why the library exists, how to install it, and the mental model: versions, the evidence-backed command database, and the layered architecture.

Part II Hands on: your first validated script in ten minutes, and the tour of the 26 curated helpers.

Part III The recipes: a literal step by step for every supported simulation type, from a single steady point to batch campaigns.

Part IV Reading results, living with versions, the real solver pitfalls (all evidence backed), and where the quality assurance confidence comes from.

How to read this guide

Every claim about solver behavior cites either the vendor manual (paraphrased, with page) or a committed probe or physics report in the repository. Nothing here is folklore.

Section outline I

1. Motivation and Installation
2. The Mental Model
3. Architecture
4. First Contact in Ten Minutes
5. The Curated Helpers
6. Recipes: Step by Step per Simulation Type
7. Reading Results
8. Versions and Evidence in Practice
9. Real Solver Pitfalls, Evidence Backed
10. Where the Confidence Comes From
11. Extending and Reference Sheets

The problem this library solves

- FlightStream is scripted through ASCII command files: one command grammar per line, executed top to bottom (SRC-003 p.279).
- The solver moves fast, with intermediate hotfix builds consolidating user requests into stable releases; the command set evolves from version to version with it.
- In script mode, failures can be quiet: a missing script file exits with code 0 and produces nothing; an invalid argument may be ignored.
- Research campaigns need hundreds of runs across solver versions, with reproducible provenance.

The failure mode

You upgrade the solver, your old script still "runs", and three weeks later you discover half the outputs were built with a command that no longer applies in the new version.

The idea in one paragraph

pyflightstream

The FlightStream version becomes an explicit input. A per-version command database, in which every entry cites either the manual page that documents its command or a committed probe report measuring that the solver accepts one no edition documents, with empirical probe evidence for every verified status, backs a script builder that refuses to emit anything invalid for the requested version. Old versions are only ever added, never dropped.

- Errors move from solver runtime (silent, cryptic) to Python build time (loud, cited).
- Every refusal message teaches: it names the physical or version cause and cites the manual page.
- The same database renders the command reference and the compatibility matrix you will use daily.

Installation and requirements

- Python 3.12 or newer (SPEC 0, computed 2026-09-09). Runtime dependencies: numpy 2.2 or newer, pandas 2.3 or newer, pydantic 2, pyyaml, trimesh, xarray.
- Install from PyPI (`pip install pyflightstream`); contributors use the editable clone below:

```
git clone https://github.com/nevesgeovana/pyflightstream.git
cd pyflightstream
pip install -e .          # library only
pip install -e .[dev]     # with tests, lint, docs tooling
```

- A licensed FlightStream installation is required only to execute scripts. Building, validating, and rendering scripts is pure Python and runs anywhere.
- The executable path is always explicit input. The library never reads it from environment variables and never guesses it.

Your offline pocket reference

- The full command reference is generated from the database and travels with the installed package:

```
import pyflightstream
pyflightstream.help()          # whole database, opens in the browser
pyflightstream.help("26.120")   # scoped to one FlightStream version
```

- The same renderer feeds the published documentation site, so the offline page and the site can never disagree.
- Each entry shows: phase, layout, typed arguments, per-version evidence status, and its evidence citation: the manual page, or a committed probe report where no edition documents the command.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

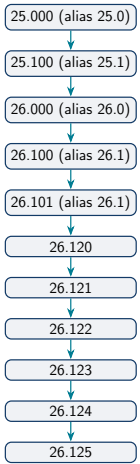
11 Extending and Reference Sheets

Versions: the YY.XXX canonical scheme

- Vendor names ("26.1", "26.12") do not sort: neither string nor float comparison orders them correctly.
- A vendor name can also name more than one build: 26.120, 26.121, 26.122, 26.123, 26.124 and 26.125 all ship as "26.12", and since 2026-08-04 so do 26.100 and 26.101 as "26.1". Resolution refuses such a name and lists the candidates, rather than returning one of them. Note what that means for a script you wrote earlier: a vendor name is unique only until the vendor ships the next build under it, so pass the canonical identifier.
- Canonical identifiers are the vendor major plus three fractional digits: `25.000`, `26.100`, `26.120`. The last digit indexes vendor hotfix builds.
- The ordered list in the database metadata is the only ordering authority, and it orders by RELEASE. Versions are never removed, and never removed is not the same as always appended: the 25 series was obtained after every 26 build and registered in front of all of them on 2026-08-09.
- Each build also carries the vendor BUILD NUMBER its solver prints, the only value that tells apart two builds sharing a release name. The generated "Which build do I have" page maps them.

Versions: resolving a name

```
from pyflightstream.versions import resolve, known_versions
v = resolve("26.120")       # canonical
v.canonical                 # "26.120"
v.build                     # "7012026", what its solver prints
resolve("26.0")             # unambiguous vendor name, accepted
resolve("26.1")             # AmbiguousVersionAliasError: 26.100
     or 26.101?
resolve("26.12")            # Ambiguous: 26.120 to 26.125, six
     builds
[str(k) for k in known_versions()]
# ["25.000", "25.100", "26.000", "26.100",
#  "26.101", "26.120", "26.121", "26.122",
#  "26.123", "26.124", "26.125"]
```



one vendor name, "26.12", names every build below 26.101; resolution refuses it and lists them

The command database and its four evidence statuses

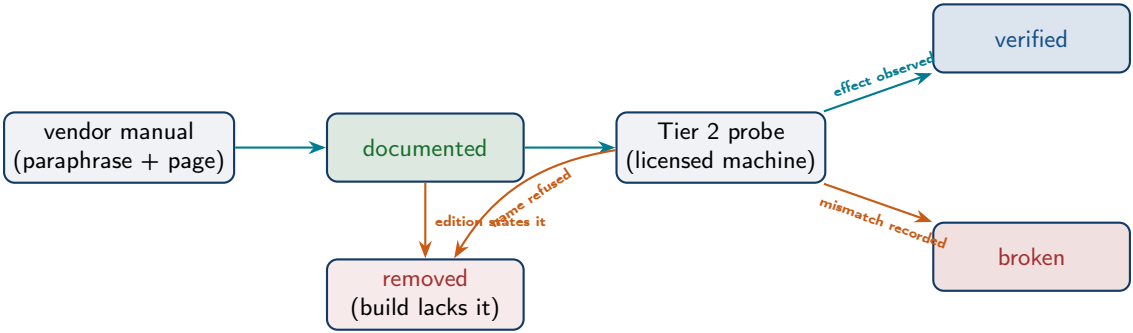
- One YAML file per manual chapter, loaded and validated at import. This guide states no command counts: the compatibility matrix is generated from the database on every docs build and publishes the current ones.
- Every entry carries: layout grammar, emission phase, typed arguments, one evidence citation (a manual page, or a committed probe report where no edition documents the command), and per-version evidence.
- Statuses are never hand edited: promotions happen only through committed probe reports.

Status	Meaning
documented	the manual says so
verified	a probe passed on a real solver
broken	a probe contradicted the manual
removed	the build does not carry it: an edition states the withdrawal, an edition stops printing it, or a probe measured the refusal

Honest emptiness

A version without recorded evidence shows an empty cell in the compatibility matrix, and the builder refuses it. No status is ever guessed.

The evidence lifecycle



promotion happens only through a committed compat report (pyfs-qa apply-compat)

- Statuses are data with provenance: `verified`, `broken` and a MEASURED `removed` all cite the report file that proved them. A `removed` read off an edition cites the page instead, and its note says which of the three ways it arrived, because the three are not equally strong.

Three tiers of quality assurance behind the statuses

Tier	What it checks	Where it runs	Evidence artifact
1	schema, builder goldens, parsers, the tier-3 matrices planned offline	CI, no solver	green test suite in continuous integration, no solver needed
2	per-command validity probes	licensed machine	`reports/compat/`
3	the workspace runs: every matrix capability as a row, the physics cases among them	licensed machine	`tests/tier3_licensed/runs.json`, `reports/physics/`

- Tier 2 asks: does this command do what the manual says, on this build? How many passed and how many were found broken is published per version by the compatibility matrix, which regenerates on every docs build.
- Tier 3 asks: does the whole workspace run, and does the physics still hold? `tests/tier3_licensed` is a campaign workspace of eleven run matrices over a synthetic library, one test per row; the physics cases are rows of it, judged against their committed references, and a cross-version drift suite compares solver builds on identical inputs.

Takeaways: the mental model

What to retain from this section

- The FlightStream version is an input, canonical form `YY.XXX`, ordered by an explicit list, never by number comparison.
- Every command the builder accepts is backed by evidence: at minimum a citation, which is the manual page or a committed probe report where no edition documents the command; at best a probe report measuring the command on a build.
- When the builder refuses, that refusal is the feature: it happens at build time, with the reason and the citation, instead of silently inside the solver.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

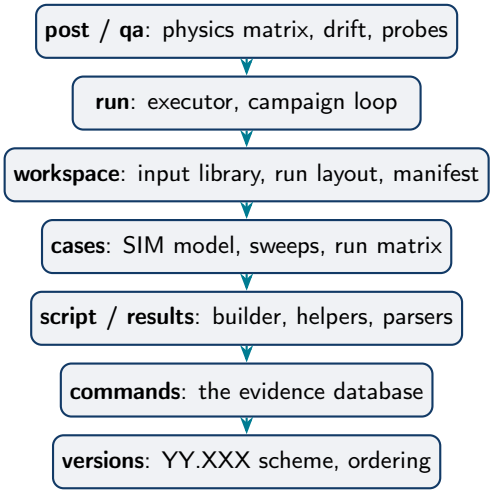
8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

11 Extending and Reference Sheets

The layered architecture



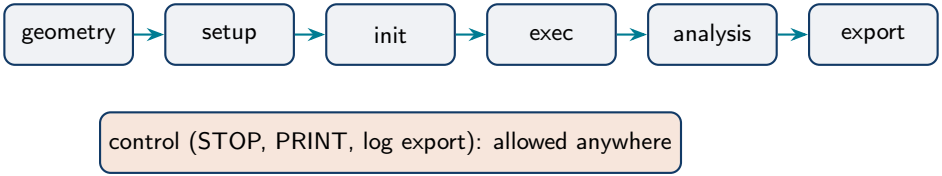
- Dependencies flow strictly downward; no layer imports upward.
- You can enter at any level of ambition:
  - one validated script (`script`),
  - one executed run (`run` + `results`),
  - a whole campaign (`cases` + `workspace`).
- The `qa` layer is how the library trusts itself; you rarely call it, but you benefit from every report it commits.

What each layer owns

Layer	Owns	Key objects
versions	canonical identifiers, ordering, aliases	`FsVersion`, `resolve`, `known_versions`
commands	per-version grammar and evidence	`CommandRegistry`, `CommandEntry`
script	validating emission, phase order, layouts, helpers	`Script`, 26 curated helpers
results	parsing solver output into typed reports	`parse_loads`, `parse_residual_history`
cases	what to run: SIM model, sweeps, recipes	`Campaign`, `SimCase`, `SweepAxis`
workspace	managed run workspace and manifest	`CampaignWorkspace`, `RunRecord`
run	how to run it: executor and campaign loop	`LocalExecutor`, `run_campaign`
qa	probes, physics matrix, drift, geometry	`pyfs-qa` CLI, `WingSpec`

- `post` (flow-visualization writers) and `fsi` (the aeroelastic coupling seam, behind the `[fsi]` extra) are delivered subpackages.

The script phase pipeline



- Every command in the database declares its phase. The builder tracks the highest phase reached and rejects a command whose phase precedes it.
- This encodes the manual's implicit ordering (import before initialize, initialize before solve) as an explicit, testable rule.
- The five layout grammars (bare, inline, param lines, payload lines, keyword block) are rendered for you; you never format a command line by hand.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 **First Contact in Ten Minutes**

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

11 Extending and Reference Sheets

Build a validated script

```
from pyflightstream.script import Script

script = Script(version="26.120")        # canonical identifier
script.comment("my first validated script")
script.emit("NEW_SIMULATION")
script.emit("IMPORT", "METER", "STL", "C:/cases/wing.stl", clear=True)
script.emit("SET_SIMULATION_LENGTH_UNITS", "METER")
script.emit("AUTO_DETECT_TRAILING_EDGES")
text = script.render()                   # complete ASCII script
```

- `emit` validates the command name, the argument types and enum values, and the phase order, against the database view of the requested version.
- `render()` returns the exact text FlightStream will read; write it to a file when you are ready to execute.

The errors are the teachers (1): typo and wrong value

```
script.emit("IMPRT", ...)
# CommandNotInVersionError: IMPRT is not in the command database.
# Nearly every command a registered manual edition documents is
# recorded, so this is usually a spelling error or a command from a
# build this install does not register; a small number an edition
# documents are not entered yet.

script.emit("SET_SIMULATION_LENGTH_UNITS", "PARSEC")
# CommandArgumentError: names the argument, lists the allowed
# enum tokens, cites the manual page.
```

Design intent

Solver-side failures are silent or cryptic; the database knows enough to fail early. Every error message names the physical or version cause and carries a citation you can check.

The errors are the teachers (2): version refusals I

```
>>> Script(version="25.0").emit("CAD_CREATE_BOX")
CommandNotInVersionError: CAD_CREATE_BOX has no recorded evidence
for FlightStream 25.000. Recorded evidence: 26.100 (documented),
26.101 (documented), 26.120 (documented), 26.121 (documented),
26.122 (documented, inherited), 26.123 (documented),
26.124 (documented), 26.125 (documented).
```

- The refusal lists exactly what is known, per version, so you can decide: switch versions, or contribute the missing evidence.
- It lists builds by REACHABILITY rather than by row, which is why a hotfix that records nothing of its own is marked inherited: a hotfix carries its base release's records wherever it has none of its own. Built from the rows alone this list hid the newest build entirely, so a caller weighing a switch was never offered the one that would have served them.

The errors are the teachers (2): version refusals II

- This sample used to refuse `NEW_SIMULATION` on 26.0, and stopped: that edition's own manual was read on 2026-08-10 and the command has a page citation there now. `CAD_CREATE_BOX` is the honest example today: the CAD primitives, the CAD body operations and the CCS mesh chapters arrive with 26.100 and are absent from every earlier build. Not the whole CAD chapter, which is what this slide used to say: the older editions do document CAD import, conversion and the curve and cross-section commands.
- Removed commands explain themselves too: the removal note, the manual page, the last documented version, and the successor command when one is recorded.

Execute and parse

```
from pathlib import Path
from pyflightstream.run import LocalExecutor
from pyflightstream.results import parse_loads

workdir = Path("C:/cases/run1"); workdir.mkdir(exist_ok=True)
script_path = (workdir / "case.txt").resolve()   # absolute!
script_path.write_text(script.render(), encoding="utf-8")

executor = LocalExecutor("C:/FlightStream/FlightStream.exe")
result = executor.run_script(script_path, working_dir=workdir,
                             timeout_s=900.0)
if not result.failed:
    report = parse_loads((workdir / "loads.txt").read_text(),
                         requested_version="26.120")
    print(report.total["CL"], report.total["CDi"])
```

- `LocalExecutor` runs the documented headless invocation (`-hidden -script`, SRC-003 pp.279-280 and RPT-023) with the working directory set for you. One dash, not two: the 25 series does not accept the two-dash spelling and does not say so, it waits for a user instead.

Takeaways: first contact

What to retain from this section

- `Script(version=...)` then `emit(...)` then `render()`: three calls stand between you and a fully validated solver script.
- Make the script path and every imported geometry path absolute; the solver resolves relative paths against its own working directory (see the pitfalls section for the evidence).
- `ExecutionResult.failed` plus `parse_loads` give you a typed, version-checked view of the run outcome.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 **The Curated Helpers**

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

11 Extending and Reference Sheets

Why helpers, and why only 26

- The database records grammar; it cannot record conditionality written in manual prose (which extras each free-stream type takes, when the initializer takes per-surface lines).
- Helpers own exactly that conditional knowledge, as thin typed functions that only call `emit`: every line still passes database validation.
- One generated function per command was rejected by design: a huge surface that teaches nothing.

The 26

free_stream, atmosphere, solver_settings, initialize_solver, start_solver, coordinate_frame, sweep, analysis_setup, export_results, probe_points, probe_line, probes_from_file, export_probes, actuator_disc, rotary_motion, unsteady_solver, azimuth_basis, blade_frames, rotate_surfaces, unsteady_action, mark_wake_edges, parse_relaxed_trailing_edge, resolve_shedding_direction, fluid_fifth_property, render_wake_edge_node_file, render_actuator_profile

Fluid state and free stream I

```
from pyflightstream.script import helpers

# ISA sea level, explicit properties (the reliable path):
helpers.atmosphere(script, density=1.225, pressure=101325.0,
                   temperature=288.15, viscosity=1.7894e-05,
                   specific_heat_ratio=1.4)

helpers.free_stream(script)              # CONSTANT (default)
helpers.free_stream(script, "ROTATION",  # hover / rotor frames
                    frame=2, axis="X", rpm=2000.0)
```

Fluid state and free stream II

Which five properties, and which unit, depend on the build

The helper reads its script's version rather than making you look it up. Builds from 26.100 on take `specific_heat_ratio` and derive the sonic velocity; the three pre-26.100 editions take `sonic_velocity` and have no specific-heat-ratio argument at all (SRC-749 p.286), so the call above is refused there and the refusal names the keyword that edition wants. The altitude form carries the sharper trap. 25.000 takes AIR_ALTITUDE with a bare value and reads it in FEET; every later build takes a units token, which this helper defaults to METERS. Same call, factor of 3.28. So on 25.000 it REQUIRES `altitude_units="FEET"` and refuses silence, that being the one omission which would change the physics without changing how the script looks.

Fluid state and free stream III

Why explicit properties

The altitude form (AIR_ALTITUDE, SRC-003 p.328) is recorded broken on 26.100, 26.101 and 26.120: the probe passed 5000 with a METERS token and the expected 0.736 kg/m^3 was not observed, with no error. ONE of the three has a reading behind it, 26.120, where 1.056 kg/m^3 was seen twice, the 5000 ft standard state and 43% high. On the other two the probe recorded only the absence; the 1.056 sentence appears in all three reports because it is the probe spec's own expectation, and it appears in 26.121's PASSING row too. So the units-ignored reading is a hypothesis with one measurement, registered as PLN-20260810-2000, and this page does not state it as three. The measurement is at ONE altitude, so what this page claims is that the METERS argument read as ignored there, and no more. The same probe passes on 26.121, and the 2026-08-08 run extended the finding backwards: the two earlier builds carry it too. The 2026-08-11 run on 26.122 passes as well (`CMP-26122_2026-08-11_full`), so the altitude form is measured to work on both hotfix builds and to fail on the three base releases it has been probed on. Three variables separate the 26.120 and 26.121 runs (build, probe harness, session file), so RPT-014 declines to credit the vendor build alone and so does this page: it is the strongest candidate for a vendor fix, and it is still a candidate. On the base releases, use FLUID_PROPERTIES.

Solver settings and initialization I

```
helpers.initialize_solver(script,
    solver_model="INCOMPRESSIBLE",   # or PRANDTL_GLAUERT, ...
    surfaces="all",
    wake_termination_x="DEFAULT",
    symmetry="NONE")                 # MIRROR, PERIODIC

helpers.solver_settings(script,
    aoa=4.0, velocity=30.0,
    ref_velocity=30.0, ref_area=8.0, ref_length=1.0,
    iterations=500, convergence=1e-5)
```

- `solver_settings` emits only what you pass, so it serves both the initial setup and re-emission between campaign points.
- Every flag argument takes a Python bool or the solver's own `ENABLE` and `DISABLE` (any case): a setup carried over from the solver speaks that vocabulary. The helper resolves it before emitting anything, and refuses a word in neither vocabulary naming the helper and the argument, so `'DISABLE'` can never reach the script as ENABLE.

Solver settings and initialization II

- Add `vorticity_drag_boundaries=[...]` to compute the induced drag of the lifting boundaries from the surface vorticity integration. Omitted on the first settings call, no command is emitted and every boundary keeps the solver's surface pressure integration (SRC-003 p.202); omitted on a later settings call of the same script, the selection of the earlier call stands. The snapshot records the effective choice either way.
- Reference area and length set the coefficient normalization ($Q\,S_{ref}$ forces, $Q\,S_{ref}\,L_{ref}$ moments; SRC-003 p.223).

Analysis selection and exports

```
helpers.analysis_setup(script,
    load_units="COEFFICIENTS")

helpers.export_results(script,
    spreadsheet="loads.txt",           # the quantitative output
    vtk="surface.vtk", vtk_variables=["CP_REFERENCE"])
```

- The induced-drag boundary selection belongs to `solver_settings` since v0.3.0 (the `analysis_setup` keyword still works and warns); the emission lands at the first curated call that reaches the analysis phase (`start_solver`, `sweep`, `analysis_setup`, or `export_results`), so a raw `script.emit("START_SOLVER")` does not flush it.
- The plain `CP` export variable is flagged for depreciation (SRC-003 p.352); the helper warns and this guide uses `CP_REFERENCE`.

Probe management I

```
helpers.probe_points(script, [(0.5, 0.0, 0.1),
                              (0.5, 0.5, 0.1)])   # simulation length units

helpers.probe_line(script, points=50,
                   start=(1.5, 0.0, 0.0), end=(1.5, 4.0, 0.0))

helpers.probes_from_file(script, "lattice.csv", units="METER")

helpers.export_probes(script, "probes_out.txt", update=True)
```

- Probes are the flow-field window: velocities and pressures at chosen points, refreshed against the current solution before export.
- The file form takes X,Y,Z,TYPE rows (0 surface, 1 off body), for whole survey lattices (SRC-003 pp.362-363).

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type
- Recipe 1: steady single point
- Recipe 2: steady polar (angle of attack sweep)
- Recipe 3: half model with mirror symmetry
- Recipe 4: actuator disc propulsion
- Recipe 5: unsteady with resolved rotating blades
- Recipe 6: probe surveys of the flow field
- Recipe 7: batch campaigns

7 Reading Results

8 Versions and Evidence in Practice

Section outline II

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

11 Extending and Reference Sheets

How the recipes are written

- Each recipe lists: prerequisites, the numbered steps (geometry, script, run, parse), the expected outputs, and a pitfalls block.
- All scripts share the same skeleton; the recipes highlight only what changes.
- Every recipe was either executed on a real 26.120 build in the repository's QA history or is assembled from verified and documented commands with citations.

The shared skeleton

NEW_SIMULATION; IMPORT geometry; length units; trailing edge and wake auto-detect; fluid state; free stream; initialize; solver settings; START_SOLVER; analysis selection; exports; CLOSE_FLIGHTSTREAM.

Recipe 1: steady single point (1/2)

Prerequisites: a surface mesh file (STL or another supported format) in known length units; the target FlightStream version; for execution, the licensed executable path.

1. Create the script and import the geometry:

```
from pyflightstream.script import Script, helpers
script = Script(version="26.120")
script.emit("NEW_SIMULATION")
script.emit("IMPORT", "METER", "STL",
            str(stl_path.resolve()), clear=True)
script.emit("SET_SIMULATION_LENGTH_UNITS", "METER")
script.emit("AUTO_DETECT_TRAILING_EDGES")
script.emit("AUTO_DETECT_WAKE_TERMINATION_NODES")
```

2. Set the fluid state and the free stream (previous section).
3. Initialize with `symmetry="NONE"` and all surfaces.

Recipe 1: steady single point (2/2)

4. Solver settings: angle, velocity, references, iterations, convergence.
5. `script.emit("START_SOLVER")`.
6. Analysis and export:

```
helpers.analysis_setup(script, load_units="COEFFICIENTS")
helpers.export_results(script, spreadsheet="loads.txt")
script.emit("EXPORT_LOG", str((workdir / "log.txt").resolve()))
script.emit("CLOSE_FLIGHTSTREAM")
```

7. Write, execute, parse (`LocalExecutor`, `parse_loads`).

Pitfalls

Geometry and script paths absolute. EXPORT_LOG needs an absolute path (relative fails silently). Verify `report.current_iteration < report.requested_iterations` to confirm convergence rather than iteration exhaustion.

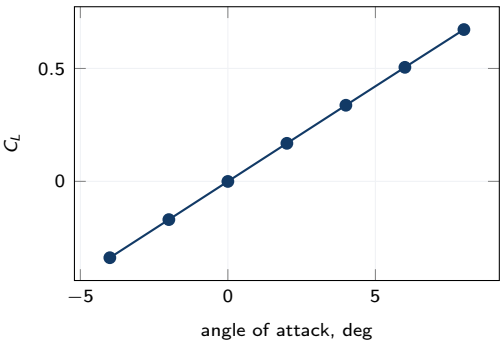
Recipe 2: steady polar, one script per point

The proven pattern (used by the repository's own physics QA): loop the single-point recipe over the angles, one script and one loads file per point.

```
rows = []
for i, alpha in enumerate([-4, -2, 0, 2, 4, 6, 8]):
    script = build_point(version, alpha, f"loads_a{i}.txt")
    path = (workdir / f"polar_a{i}.txt").resolve()
    path.write_text(script.render(), encoding="utf-8")
    result = executor.run_script(path, working_dir=workdir)
    report = parse_loads((workdir / f"loads_a{i}.txt").read_text(),
                         requested_version=version)
    rows.append((alpha, report.total["CL"], report.total["CDi"]))
```

- Fully restartable: each point is independent evidence with its own log.
- This is exactly `examples/steady_polar.py` in the repository, which also demonstrates the version refusal and the finite-wing sanity check.

Recipe 2: what a healthy polar looks like



- Real data: NACA 0012 rectangular wing, aspect ratio 8, FlightStream 26.120, from the committed example run.
- Lift slope 4.83 per rad; finite-wing theory anchor $2\pi/(1+2/AR) = 5.03$ per rad.
- A grossly wrong slope points at a broken import or symmetry setup, not at solver drift: cheap sanity first.

Recipe 2 variant: the Sweeper Toolbox

When to prefer it: one solver session sweeping internally, reusing the solution between points (faster, but one shared session and one shared export).

```
helpers.sweep(script,
    aoa=[-4.0, -2.0, 0.0, 2.0, 4.0, 6.0, 8.0],
    clear_solution=False,
    export_spreadsheet="sweep_results.txt")
```

Pitfalls

The Sweeper chapter was redrafted from its reference pages (SRC-003 pp.358-360) on 2026-08-06, replacing entries first read from the worked example at p.406. That example exercises two of the three sweep modes and three of the four axes, so the earlier entries were missing the UNIFORM mode, the Mach axis, and the inline value list on the velocity axis. What the database still does not enforce is which tail each mode takes: the emitter accepts UNIFORM with no numbers, or DISABLE with numbers, and the solver is the first thing to object. For evidence-critical campaigns, prefer one script per point.

Recipe 3: half model with MIRROR symmetry I

Prerequisites: a true half geometry with an open root section lying in the symmetry plane. A face on the symmetry plane duplicates itself into coincident faces (SRC-003 p.386, paraphrased).

1. Import the half mesh; same skeleton as recipe 1.
2. Initialize with mirror symmetry:

```
helpers.initialize_solver(script, symmetry="MIRROR")
```

3. Keep the FULL planform reference area, then request the symmetry loads explicitly:

```
helpers.solver_settings(script, ref_area=full_planform_area, ...)
helpers.analysis_setup(script, symmetry_loads=True,
                       load_units="COEFFICIENTS")
```

Recipe 3: half model with MIRROR symmetry II

Pitfalls (all evidence backed)

A nonzero sideslip under MIRROR is refused at plan time since 0.13.1: the solver was measured (26.120) running a mirrored half model at zero sideslip whatever the script states; sweep sideslip on a full geometry with SYMMETRY NONE. Initializing MIRROR with a full model diverges instantly (SRC-003 p.217, paraphrased). The post-MIRROR symmetry loads default was calibrated on the real solver as ENABLE, but emit it explicitly: with DISABLE the loads halve and a one-wing rolling moment appears. Expected equivalence versus the full model: $\Delta C_L \approx +0.0015$, $\Delta C_{Di} = 0.0$ (committed physics report).

Recipe 4: steady installed propulsion with an actuator disc (1/2)

Concept: the disc is the linearized propeller slipstream surrogate (SRC-003 pp.185-187); steady, cheap, no blade geometry.

1. Build the airframe skeleton as in recipe 1, up to the fluid state.
2. Create a local coordinate system carrying the disc axis (frame index greater than 1):

```
script.emit("CREATE_NEW_COORDINATE_SYSTEM")
script.emit("SET_COORDINATE_SYSTEM_ORIGIN", 2, x, y, z)
# orient axis as needed: SET_COORDINATE_SYSTEM_AXIS ...
```

3. Create the disc through the helper (returns the actuator index):

```
disc = helpers.actuator_disc(script, "prop1",
    frame=2, axis="X", offset=0.0,
    r_tip=0.95, r_hub=0.15, rpm=2200.0,
    thrust=850.0, thrust_type="NEWTONS")   # ELLIPTICAL model
```

Recipe 4: steady installed propulsion with an actuator disc (2/2)

4. Alternative thrust specification: a per-blade radial profile file (CUSTOM model), which then requires the blade count:

```
disc = helpers.actuator_disc(script, "prop1",
    frame=2, axis="X", offset=0.0, r_tip=0.95, r_hub=0.15,
    rpm=2200.0, profile="radial_thrust.txt", n_blades=6,
    swirl=0.8)   # fraction of swirl kept; below 1 mimics a stator (SRC-003 p.186)
```

5. Initialize, set solver settings, START_SOLVER, analyze, export: identical to recipe 1.

Pitfalls

Exactly one thrust specification: net thrust or profile, never both. The manual recommends dimensional thrust over the coefficient form, because the coefficient convention must match the solver formulation (SRC-003 p.187). The helper enforces both rules with cited errors. The profile file is rows `r,F` with no header and no final newline: 26.124 reads a final newline as one point more and refuses the file in a dialog (RPT-070). Pass the file's text as `profile_text` and the helper puts it in that form, through `render_actuator_profile`, for the run to write where `profile` names; a matrix row's run does this for you.

Recipe 5: unsteady rotary motion (1/2) I

Concept: the blade-resolved alternative to the actuator disc (SRC-003 p.234): the blade geometry physically rotates through unsteady time stepping. Costlier, richer.

1. Import the full geometry including blades. Their names come with the mesh, and naming them is preferred to counting them: a position is right for one file's boundary order and means another surface in a file that orders them differently.
2. Create the rotation frame (as in recipe 4), then the motion:

```
script.declare_existing(boundaries={   # the names the mesh carries
    "Blade1": 3, "Blade2": 4, "Blade3": 5,
    "Blade4": 6, "Blade5": 7, "Blade6": 8})
motion = helpers.rotary_motion(script,
    frame=2, axis="X", rpm=2200.0,
    boundaries=["Blade1", "Blade2", "Blade3",
                "Blade4", "Blade5", "Blade6"],
    wake_stabilization_blades=6)
```

3. Select the unsteady solver with the time step sized from the manual guidance: 8 to 12 degrees of blade rotation per step, at least two full rotations (SRC-003 p.210).

Recipe 5: unsteady rotary motion (1/2) II

```
# 2200 rpm = 13200 deg/s; 10 deg/step -> dt = 7.58e-4 s
# two rotations = 720 deg -> 72 steps
helpers.unsteady_solver(script, time_iterations=72,
                        delta_time=7.58e-4)
```

Recipe 5: unsteady rotary motion (2/2) I

4. For rotor-dominated velocity scales, normalize with the tip speed instead of the free stream (SRC-003 p.201):

```
tip_speed = 2200.0 / 60.0 * 2 * 3.14159 * r_tip
helpers.solver_settings(script, velocity=v_inf,
                        ref_velocity=tip_speed, ...)
```

5. START_SOLVER, then analysis and export as usual; the loads spreadsheet reflects the final time step.

Recipe 5: unsteady rotary motion (2/2) II

Pitfalls

On 25.100, 26.000 and 26.100 the motion is written as a Euclidean motion and `wake_stabilization_blades` is refused by name, so drop it there; on 26.100 the rotor carries no rotor mark and its speed is written in rev/min, unmeasured (the per-build table of the workflow page, RPT-051). SET_MOTION_START_TIME is recorded broken on 26.101, 26.120 and 26.121, and 26.122 inherits 26.120, so the builder refuses it on four builds and you must converge the steady base flow by other means for now. But the abort was the PROBE's doing rather than the solver's, and this note says so because the status cannot yet be taken back. The family's shared probe prelude created a rotary motion, which is the one kind whose start time the solver declines to set; with a 6DOF prelude the command runs on all four builds (RPT-026 and the `CMP-<build>_2026-08-11_starttime` reports for 26.101, 26.120 and 26.121, and `CMP-26122_2026-08-11_full` for the newest). An unobservable effect is not evidence that a command works, so no sanctioned path demotes the status on it.

Recipe 6: probe surveys

Concept: extract velocities and pressures at chosen stations: wake rakes, slipstream surveys, control surfaces for conservation budgets.

1. Run any of recipes 1 to 5 up to and including START_SOLVER.
2. Define the stations (three equivalent entries):

```
helpers.probe_points(script, [(1.5, 0.0, 0.0)])   # simulation length units
helpers.probe_line(script, points=50,
                   start=(1.5, -2.0, 0.0), end=(1.5, 2.0, 0.0))
helpers.probes_from_file(script, "survey_lattice.csv",
                         units="METER")   # X,Y,Z,TYPE rows
```

3. Export with refresh, so the values reflect the converged solution:

```
helpers.export_probes(script, "probes_out.txt", update=True)
```

Pitfalls

Keep `update=True` unless you deliberately want stale values; the export otherwise reflects whatever refresh happened last.

Recipe 7: campaigns (1/3), declare what to run I

Concept: a campaign is data (TOML), a recipe is code (a function), the loop welds them with full provenance.

```
[campaign]
name = "wing_steady_sweep"
fs_version = "26.120"
fs_exe = 'C:\FlightStream\26.12\FlightStream.exe'

[[sim]]
sim_id = "9001"
aircraft = "TestWing"
description = "steady polar"
reynolds = 4.38e6
mach = 0.1441
sweep = {type = "alpha", values = [-4.0, 0.0, 4.0, 8.0]}
recipe = "my_recipes:steady_point"
outputs = ["loads_{point}.txt"]
```

- Sweep axes: any key of `FLIGHT_CONDITION` since 0.21.0, which is the flow variables, the two angles, the advance ratio, `RPM` and the three body rates; the Python `SweepAxis` type also takes the paired `alpha_beta` of the matrices written before 0.15.0.

Recipe 7: campaigns (1/3), declare what to run II

- Output names carry `{point}`: every point of a case runs in the same simulation folder, so a constant name would let each point overwrite the previous one's evidence. A multi-point case that declares a constant name is refused before anything runs.

Recipe 7: campaigns (2/3), write the recipe once

```python
# my_recipes.py
from pyflightstream.cases import SimCase
from pyflightstream.script import Script, helpers

def steady_point(case: SimCase, script: Script) -> None:
    script.emit("NEW_SIMULATION")
    script.emit("IMPORT", "METER", "STL",
                str(case.geometry), clear=True)
    ...  # skeleton as in recipe 1
    helpers.solver_settings(script,
        aoa=case.point["alpha"], ...)
    helpers.export_results(script,
        spreadsheet=case.outputs[0])
    script.emit("CLOSE_FLIGHTSTREAM")
```

- The loop fills `case.point` per sweep point, stages the geometry, and renders the declared output names for that point into `case.outputs`; export what it hands you and evidence lands inside the managed simulation folder, one file per point.
- Recipes are imported functions, resolvable by name registry or `module:function` reference: explicit and versionable.

Recipe 7: campaigns (3/3), run with provenance

```python
from pyflightstream.cases import load_campaign
from pyflightstream.workspace import CampaignWorkspace
from pyflightstream.run import (LocalExecutor, run_campaign,
                                LoadsAssessor)

campaign = load_campaign("campaign.toml")
records = run_campaign(campaign,
    executor=LocalExecutor(campaign.fs_exe),
    workspace=CampaignWorkspace("C:/cases/campaign_root"),
    assess=LoadsAssessor())   # judges whichever collected
                              # output reads as a loads table
```



- Every executed point lands in exactly one of six statuses; a silent skip is structurally impossible. The manifest (`runs.json`, append only, with staging hashes) is the run identity: names lie, hashes do not.

The flight condition: one cell, and where its keys may live I

- From v0.9.0 a row states its whole flow condition in one `FLIGHT_CONDITION` cell. **It is a set of CONSTRAINTS on one flow state, and the keys you give decide which quantity gets solved for.** It is not a record with optional fields.

```
FLIGHT_CONDITION: MACH:0.20, REmi:5.5
FLIGHT_CONDITION: TASmps:68.08, ALTFT:10000, dISA:5
```

- The first states a Mach and a Reynolds number and no temperature, so the state is sea level and **the density moves** to meet the Reynolds number. The second states a speed at an altitude, so it is an atmosphere point and **the Reynolds number comes out** of it.

Key	Unit	What it constrains
`MACH`	dimensionless	velocity, via the speed of sound at the state's own temperature
`TASmps`	metres per second	velocity directly (true airspeed)
`REmi`	**MILLIONS**: `5.5` is 5 500 000	density
`ALTFT`	**feet**	pressure, and temperature via the standard lapse
`dISA`	**Celsius, as a DELTA**	temperature, as an offset on the standard

The flight condition: one cell, and where its keys may live II

- **Read the units off the key names.** They are ugly on purpose: `ALTITUDE:10000` would be ambiguous between feet and metres. `REmi` is the one to get right, because a Reynolds number written in absolute units instead of millions is wrong by a factor of a million and still RESOLVES: it produces a density, emits, solves and reports.
- State **exactly one** of `MACH` or `TASmps`: neither is under-determined and both is a contradiction, and each is refused. `REmi` needs the row's `REF`, because a Reynolds number has no meaning without a length. The set is CLOSED, so an unknown key is refused rather than ignored.
- **Five further keys PIN the fluid** (v0.11.0): `RHOkgm3`, `MUPas`, `ASMPS` (speed of sound), `TK` and `PPA`, each replacing what the standard atmosphere would supply and each recorded as pinned. From v0.12.0 they may instead live in the setup artifact the row's `SET` names, under a `[flight_condition]` table, so a thirteen-point polar states them once rather than thirteen times; a row that states one wins over the setup's. The velocity keys and `REmi` may NOT go there, because they are what makes a point that point.
- Full grammar, every refusal and the migration: `docs/flight-conditions.md`.

Every input key, in one generated page

- `inputs/INPUTS.md` lists every key an input artifact may state: the matrix row by run type, the setup, the pproc, the reference and the geometry sidecar. Each row says what the key sets, its unit or values, the run types or builds that accept it where the code says, and the solver command it reaches.
- It is **generated from the code**, at the root of `inputs/`, by `pyfs-workspace init`, `pyfs-matrix plan` and `post`, and rewritten only when its content changes; the documentation site carries the same page. A key the package gains arrives there with its meaning, and a test fails for a key that arrives without one.
- Beside it, `inputs/input_template.md` (v0.28.0) shows every input FILE: one section per kind, the matrix, the setup, the pproc, the reference, the geometry sidecars, the files of `profiles/`, the free stream, the HPC profile and the build registry, each with a complete example to copy to the path its title names. Every example is a file the package reads as it stands, which the suite checks by reading each one back.

Recipe 7 addendum: migrating a run matrix I

- Coming from spreadsheet-driven run matrices (the pipe-delimited layout with a `SWEEP_TYPE` column of `AL` and `BE` codes)? **First upgrade the file.** The layout has broken four times and a matrix written under any earlier one is recognised by its header and refused rather than misread. v0.9.0 replaced the `RE` and `MACH` columns with one mandatory `FLIGHT_CONDITION` cell; v0.15.0 removed `SWEEP_TYPE` itself, because that cell now says which variable varies by carrying the word `sweep` where its value would be, so `AL` becomes `ALPHA:sweep`. The one row the converter will not do for you is one that varies BOTH angles: that is one row per sideslip, each needing a `POL` of its own, and it is refused by name.

```
pyfs-matrix upgrade matrix.fs --in-place
# without --in-place it goes to standard output, so you
# can diff before you overwrite. It needs no recipes, no
# version and no executable.
```

Recipe 7 addendum: migrating a run matrix II

- **The values move across verbatim; the RESULT may not.** Under v0.8.x the `RE` column was recorded metadata that reached no emitted line. From v0.9.0 `REmi` is a CONSTRAINT that solves for density, so an upgraded row emits an explicit fluid state it never emitted before and the numbers will differ. To keep the previous behaviour, state the condition without `REmi` and let the Reynolds number be derived.
- Then the reader converts:

```
from pyflightstream.cases.matrix import convert_matrix
text = convert_matrix("matrix.fs", name="wing2026",
    fs_version="26.120", fs_exe="C:/fs/FlightStream.exe",
    recipes={"003": "recipes.steady_polar:build"})
# the matrix codes are preserved as matrix_* variables,
# nothing is dropped; the TOML round-trips through load_campaign
```

- One-command form: the `pyfs-matrix convert` CLI takes the same inputs as flags and writes the file via `-o campaign.toml`.
- Convert once, review the TOML in a diff, then live natively in the campaign format.

Recipe 7 addendum: migrating a run matrix III

- Conversion carries the output names too: a row declares them in `VAR_NAMES_VALUES` as `OUTPUTS: loads_{point}.txt`, comma-separated because the slash already separates the KEY:VALUE pairs, and `convert_matrix` writes them into the converted `[[sim]]`. A row declaring none is refused before the solver starts.

Recipe 7 addendum: migrating a loose script builder I

The other half of coming from a driver script: the campaign loop calls the `ScriptRecipe` protocol, `build(case, script) -> None`. A builder with the old signature is refused by `resolve_recipe`, so `pyfs-matrix plan` blocks every point of the case before anything runs, naming the signature it found.

```
# before: the loose form an old driver called directly
def build_polar(workdir) -> Script:
    script = Script(version="26.120")
    script.emit("OPEN", r"C:\cases\wing.fsm")
    helpers.solver_settings(script, aoa=4.0, ...)
    helpers.export_results(script,
        spreadsheet=str(workdir / "loads.txt"))
    return script

# after: the protocol the managed loop calls
def build_polar(case: SimCase, script: Script) -> None:
    script.emit("OPEN", str(case.geometry))
    helpers.solver_settings(script,
        aoa=case.point["alpha"], ...)
    helpers.export_results(script,
        spreadsheet=case.outputs[0])
    script.emit("CLOSE_FLIGHTSTREAM")
```

Recipe 7 addendum: migrating a loose script builder II

- **Do not create the Script.** The campaign owns it, already bound to the campaign's FlightStream version; emit into the one you are given and return None.
- **Take the geometry from the case.** The loop rewrites `case.geometry` to the staged copy it hashed, so opening it opens exactly the file the manifest names.
- **Take the point from the case.** A hard-coded angle becomes `case.point["alpha"]`; the loop fills `case.point` per sweep point.
- **Export the names you were handed.** An absolute `workdir` path becomes `case.outputs[i]`: the loop renders the declared names for the point being built, and collects exactly those. Every point of a case runs in the same folder, so a constant name would overwrite the previous point's evidence, and declaring one is refused.
- **Give it an address.** The function must live in an importable module and be named in the case as `"package.module:function"`; that string is what the loop resolves.

Takeaways: choosing your recipe

You want	Recipe	Cost
one force and moment set	1: steady point	minutes
a lift or drag polar	2: polar (per point)	minutes per point
symmetric config at half cost	3: MIRROR half model	like 1
installed propulsion, steady	4: actuator disc	like 1
blade-resolved slipstream	5: unsteady rotary	hours
flow-field quantities	6: probes, on top of any	marginal
hundreds of runs, provenance	7: campaign	scales linearly

Composition

The recipes compose: a campaign whose recipe function builds an actuator disc case with a probe survey is just recipe 7 wrapping recipes 4 and 6.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 **Reading Results**
- **What every product carries from 0.23.0**

8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

Section outline II

11 Extending and Reference Sheets

The loads report

```
report = parse_loads(text, requested_version="26.120")
report.total          # {"CL": ..., "CDi": ..., "CDo": ..., "CMy": ...}
report.surfaces       # per-boundary rows, keyed by name
report.current_iteration, report.requested_iterations
report.fs_version_reported, report.fs_build   # solver identity
```

- Convergence check: `current_iteration < requested_iterations` means the residual threshold was met before iteration exhaustion.
- The parser is strict about structure (a table without its closing Total row raises) and lax about version prefixes: the 26.120 build reports itself as "26.1", so the check records the reported string and build verbatim and warns on true mismatches.

The residual history

```
from pyflightstream.results import parse_residual_history
samples = parse_residual_history(log_text)
# [ResidualSample(iteration=..., residual=...), ...]
```

- The residual trace is the difference between "it converged" and "it stopped": plot it when a result surprises you.
- Divergence shows as growth; a plateau above the threshold shows a case that needs more iterations or a better wake setup.
- The campaign assessor uses exactly these signals to separate CONVERGED from COMPLETED_MAX_ITER from FAILED_DIVERGED.

Every product says what it is a file OF

```
POLAR,DESCRIPTION,GROUP,SREF,CREF,BREF,XMOM,YMOM,ZMOM,VINF,ALT,J,ALPHA,...
PROBE,X,Y,Z,FRAME,STEP,ALPHA,BETA,MACH,RE,VINF,ALT,J,SREF,CREF,BREF,...
ITERATION,AZIMUTH,ALPHA,BETA,MACH,RE,VINF,ALT,J,SREF,CREF,BREF,Offset,...
```

- From 0.23.0 every file the post stage writes carries the WHOLE flight condition and the reference lengths. Before it, the sections, probes and reduction tables carried no reference length at all.
- A coefficient beside no reference area is a number nobody can check, and a probe sample with no condition is a table about nowhere.
- `_sections` now leads with `ITERATION` and `AZIMUTH`: what varies down the table. It used to lead with the polar's NAME, which the file name already carries.

No cell is ever blank

```
from pyflightstream.post.products import NOT_APPLICABLE   # "NA"
```

- A cell that does not apply to a row reads `NA`. A blank is ambiguous three ways: zero, not measured, or this column is not for this row.
- **pandas readers see no change**: both `''` and `'NA'` are default missing tokens. A hand parser that tested `cell == ""` must test for `NA`.
- One token, not two: the probes `STEP` column wrote `-` until 0.23.0. The PRINTED plan table keeps `-` deliberately, because nothing parses it.

Rotor coefficients, one set per rotor

```
J_PUSHER, CT_PUSHER, CQ_PUSHER, CP_PUSHER, ETA_PUSHER, ETAW_PUSHER

J  = V/(|n|D)       CT = T/(rho n^2 D^4)   CQ = Q/(rho n^2 D^5)
CP = 2 pi CQ sign(n)   ETA = J CT / CP     ETAW = J CTW / CP
```

- `n` is signed revolutions per SECOND: its magnitude sets `J`, its rotation sign multiplies `CP`, and `CQ` keeps the torque sign about the fixed rotor axis. `CTW` is the rotor's force projected on the free stream, against it, over `rho n^2 D^4`. `ETAW` equals `ETA` when the shaft lies along the stream.
- Every column carries the rotor's ALIAS, because these make physical sense for ONE rotor and not for several summed: the diameters and speeds that normalise them are different numbers.
- `ETA` and `ETAW` read `NA` on a static point, where both are 0/0. A figure of merit is the static measure and you define it.
- The table names its alias in its `ROTOR` column, right after `POL`, on every row, so a script that has already loaded the file still knows which rotor and which polar it holds. From 0.23.0 to 0.26.x the alias stood alone on the first line, before the header; since 0.27.0 the first line of every table is its header and its first column the polar.

A rotor states how it is INSTALLED

```
[PUSHER]
kind = "rotor"
axis = "Z"                      # as before: Z IS (0, 0, 1)
axis = [0.0, 0.199, 0.980]      # a mesh that arrives with pitch and toe in it
```

- Until 0.23.0 every frame the package built for a rotor used the geometry's own axes, which assumes the rotor is installed at zero pitch and zero toe.
- A LETTER keeps its exact meaning and takes the same code path, so every reference written before this release emits the same script.
- The blade datum is refused by ANGLE when it lies within five degrees of the shaft. The old check compared two strings, so it was blind to a datum one degree from parallel, whose azimuth locates nothing.
- **Not yet validated against a licensed run**: no run with pitch and toe exists yet. What is proved offline is that the letter path is unchanged.

Unsteady products come from the plots

```
from pyflightstream.cases.windows import averaging_span
# the row states LAST_REVS_AVG = 0.5; the run took 1000 steps, 250 a revolution
averaging_span({"LAST_REVS_AVG": "0.5"}, last_step=1000, per_revolution=250.0)
# (876, 1000)
```

- The POLAR, the rotor table and `per_blade` all average over the SAME window: the LAST revolutions (`LAST_REVS_AVG`) or iterations (`LAST_ITERS_AVG`) the matrix row states, ending at the run's last step. Two windows would put a difference in the fourth digit that nobody could attribute.
- `per_blade` is ONE window, one row per blade, with each blade's start and end azimuth in columns. It averaged each blade over its own passage before, which mixed a real azimuthal difference with a difference in WHEN each blade was sampled.
- The native coefficient export stays as a HEALTH CHECK only. It is the last iteration, which on an oscillating rotor is one instant of a cycle, so the POLAR no longer reads it and the assessor judges from the history.

Groups are named, and your files are renamed for you

```
from pyflightstream.workspace import rename_group_products
rename_group_products(root, {1: "PUSHER"}, dry_run=True)   # look first
rename_group_products(root, {1: "PUSHER"})
```

- A product file carries the group's NAME instead of _g01. A number told you which position the group held in a list, which is a fact about the list and not about the group.
- You supply the mapping: nothing in a workspace records which group _g03 was. A number you do not give is LEFT ALONE, never renamed on a guess.
- It ARCHIVES each file before it moves it, and never deletes. A destination that already exists is refused, naming both paths, with nothing moved.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 **Versions and Evidence in Practice**

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

11 Extending and Reference Sheets

Reading the compatibility matrix

- The docs site renders every command against every registered version; the same data backs `pyflightstream.help()`.
- The page opens with an evidence summary per version, counting verified, documented, broken, removed and no-evidence, and naming the manual edition each version was read from. Those counts are generated, never restated here: a number written by hand in this guide goes stale on the next licensed session, and has.

When the builder refuses your version

1. Check the matrix: is the command recorded for any version?
2. If the manual for your version documents it, add the documented line with the page citation (contribution path).
3. On a licensed machine, run the Tier 2 probe to promote it to verified.

Hotfix inheritance I

- A hotfix build inherits the evidence record of its base release until probe evidence overrides it, WHEN the registry says it does. The last canonical digit does not decide that: `26.101` sits at a hotfix index behind `26.100` and is an independent vendor release, so it inherits nothing. Each build states `inherits_base` in the registry and one that states nothing is refused.
- `26.121` is no longer the hypothetical example: it started from every `26.120` record, and the licensed probe on that build then contradicted one of them. AIR_ALTITUDE is broken on 26.120 and verified on 26.121. The 2026-08-08 run then probed the two earlier builds for the first time and found the defect there as well, and the 2026-08-11 run on 26.122 found the altitude form working there too (`CMP-26122_2026-08-11_full`), and the 2026-08-17 run on 26.123 found it working there too, so the hotfixes it has been probed on are the builds it is measured to work on and the three base releases it has been probed on are the builds it is measured to fail on. The matrix carries the list; a tally written here has gone stale on every registration.
- Read an inherited cell for what it is. The matrix marks every one with a base superscript and counts them in its *Of which inherited* column, so a marked cell means the base release's evidence, with no probe on this build having run that command.

Hotfix inheritance II

- Inheritance is switched OFF in both directions at once, and only the first is obvious. A build inheriting nothing loses the base's *verified* records, which is the cost everyone counts; it also loses the base's broken ones. `SET_MOTION_START_TIME` is broken on 26.101, 26.120 and 26.121 on those builds' own evidence, and on 26.123 it carries a *documented* row from a reading with no probe, so the emitter allows it there. A command refused on four builds and emitted on the fifth is not a statement that the fifth fixed it.

Why the identifiers carry the hotfix digit

Vendor hotfix builds can change solver behavior, and here the change was a silent numerical one: the same command and the same assertion, a density 43% apart. NOT the same script, which is why this page names no cause: the probe harness moved between the two runs, so they are not a controlled comparison (RPT-014). What the hotfix digit buys is that the two builds keep separate records, so the disagreement is visible at all instead of one answer quietly standing for both.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 Versions and Evidence in Practice

9 **Real Solver Pitfalls, Evidence Backed**

10 Where the Confidence Comes From

11 Extending and Reference Sheets

Pitfalls found by the probe suite (Tier 2) I

Finding	Builds	Practical rule
AIR_ALTITUDE does not produce the altitude asked for	26.100, 26.101, 26.120	set the fluid state through FLUID_PROPERTIES, or move to a hotfix build where the same probe passes; the compatibility matrix names which
SET_MOTION_START_TIME aborts the script	26.101, 26.120, 26.121, 26.122*	READ THE THIRD NOTE BELOW BEFORE ACTING ON THIS ROW
NEW_OFF_BODY_STREAMLINE aborts the script	26.120, 26.121, 26.122, 26.123, 26.124, 26.125	use probe lattices for off-body data

Pitfalls found by the probe suite (Tier 2) II

Finding	Builds	Practical rule
SWEEPER_REF_VELOCITY_SAME aborts the script	26.121	prefer one script per sweep point
SOLVER_TIME_AVERAGING hangs the solver before it marches	26.124	never emitted since 0.28.0: the pproc's `[time_averaging]` makes the package average the per-step surface exports itself, the average the solver computes on 26.122 (RPT-079)

Finding	Builds	Practical rule
EXPORT_BL_VELOCITY_PROFILE holds an unattended script	26.124	the boundary-layer thicknesses come from the VTK surface; the profile needs a person at the machine (RPT-075)
NEW_CCS_FUSELAGE_RELAXED_TE aborts the script	26.124	the CCS mesh recipe that needs it stops at it; leave it out of a script until a re-probe clears it, and the builder refuses it meanwhile (RPT-126)
NEW_CCS_REVOLVE_RELAXED_TE aborts the script	26.124	the CCS mesh recipe that needs it stops at it; leave it out of a script until a re-probe clears it, and the builder refuses it meanwhile (RPT-126)

Pitfalls found by the probe suite (Tier 2) III

Finding	Builds	Practical rule
NEW_CCS_WING_FLAP_COVE aborts the script	26.124	the CCS mesh recipe that needs it stops at it; leave it out of a script until a re-probe clears it, and the builder refuses it meanwhile (RPT-126)
NEW_CCS_WING_MORPHING_SURFACE aborts the script	26.124	the CCS mesh recipe that needs it stops at it; leave it out of a script until a re-probe clears it, and the builder refuses it meanwhile (RPT-126)
EXPORT_REVOLVE_CCS_FILE aborts the script	26.124	the CCS mesh recipe that needs it stops at it; leave it out of a script until a re-probe clears it, and the builder refuses it meanwhile (RPT-126)

Pitfalls found by the probe suite (Tier 2) IV

Finding	Builds	Practical rule
EXPORT_FUSELAGE_CCS_FILE writes no file that was observed	26.124	the command ran but the file it names was not observed; do not rely on the export, and the builder refuses it meanwhile (RPT-126)
EXPORT_WING_CCS_FILE writes no file that was observed	26.124	the command ran but the file it names was not observed; do not rely on the export, and the builder refuses it meanwhile (RPT-126)
CREATE_FREE_SURFACE_TFI_MESH aborts the script	26.125	the documented call raised a modal error naming a perimeter curve with no virtual curves assigned; what the free surface needs first is not measured, so leave it out of a script, and the builder refuses it meanwhile

Pitfalls found by the probe suite (Tier 2) V

- Each carries **broken** status for the builds named, with the committed report cited, and the builder REFUSES to emit them for those builds (`BrokenCommandError`, FR-48): a broken command is one a probe watched fail, and unlike an absent one it produces a complete run with wrong numbers in it. If a run needs one anyway, and re-probing it is the run that does, `script.allow_broken(NAME, reason=...)` emits it and records the command, the report, your justification and TWO builds: `version` (targeted) and `source_version` (where the cited report ran).
- Read the last row precisely, because documented does not mean never run. The 2026-07-21 sweep DID execute SWEEPER_REF_VELOCITY_SAME on 26.120, without a script abort, and left it unpromoted because its effect is not observable with the current instruments. It aborts on 26.121. So this is an outcome change rather than a first measurement, and it is still not attributable to the build: three variables separate the runs (RPT-014 section 4.3).
- 26.122 was measured on 2026-08-11 and the run moved two of these rows. AIR_ALTITUDE leaves the table for that build: it had inherited **broken** from 26.120, and the probe reads the 5000 m standard-atmosphere density correctly, so the inherited status was PESSIMISTIC and this table was about to send you away from a command that works. NEW_OFF_BODY_STREAMLINE keeps its row and loses its asterisk: it is now measured on 26.122 rather than inherited, and it fails harder than the word "aborts" suggests, with an access violation.

Pitfalls found by the probe suite (Tier 2) VI

- The asterisk marks inheritance, as everywhere else in this table, and the row it is left on additionally carries a warning. SET_MOTION_START_TIME is recorded broken on 26.101, 26.120 and 26.121, and inherited by 26.122 from 26.120, so the builder refuses it on all four and the row is still printed; but the abort was the PROBE's doing, not the solver's. The shared prelude of that command family creates a rotary motion, and a rotary motion is the one kind whose start time the solver declines to set (measured in RPT-026). With the prelude corrected the command runs without aborting on 26.101, 26.120 and 26.121 (reports `CMP-26101_2026-08-11_starttime`, `CMP-26120_...` and `CMP-26121_...`) and on 26.122 (`CMP-26122_2026-08-11_full`, which is the full run rather than a starttime report). The statuses cannot be taken back yet, because an unobservable effect is not evidence that a command works and no sanctioned path demotes on it. Treat this row as pending rather than as a finding.
- (NEW_SURFACE_SECTION_DISTRIBUTION was in this list until the 2026-07-23 licensed re-probe promoted it to **verified** on the corrected phase and grammar.)

Pitfalls found by real runs (Tiers 2 and 3)

- **Silent exit 0.** If `-script` names a missing file, the solver exits with code 0 and zero outputs. Always resolve the script path absolute, and treat "no outputs" as failure (the campaign loop does).
- **A command line the build does not know is a hang, not an error.** Give a 25.x build the two-dash spelling and it starts, checks out its licence, receives no script and waits; windowless with the streams redirected, the Fortran runtime then opens a modal dialog nobody can answer. A clean licence checkout is not evidence that a run reached the script (RPT-023).
- **Relative paths resolve against the solver working directory.** Imported geometry: absolute. EXPORT_LOG: absolute, or it fails silently.
- **The hidden solver idles after STOP.** A headless session that hits STOP does not exit; the executor's timeout is your safety net, and the log pair is the halt evidence.
- **Zero induced drag is not always good news.** A boundary without a user-defined trailing-edge condition, a bluff body such as a tubular fuselage or a pod, reports zero induced drag once it is placed on the vorticity CDi list; left off the list it keeps the surface pressure integration instead (SRC-003 p.202). This is why `vorticity_drag_boundaries="all"` is unsafe on a mixed geometry.
- **The transitional boundary layer model has a validity window.** Stated valid for chord Reynolds numbers between 500000 and 1500000 (SRC-003 p.203); outside it, select LAMINAR or TURBULENT explicitly.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 **Where the Confidence Comes From**

11 Extending and Reference Sheets

The physics anchors

- The committed physics matrix runs a synthetic NACA 0012 wing (generated from code, reproducible by anyone) against banded references:
  - PHY-01, the polar: per-angle C_L, the lift slope against the finite-wing anchor, induced drag at the reference angle.
  - PHY-02, the symmetry equivalence: the mirrored half model must reproduce the full-span coefficients on the same full planform reference.
- References change only through a tool that demands a written reason, and a reference update never shares a commit with code changes.
- Repeat runs on the same build are bit identical; that is the baseline that makes drift detection meaningful.

Case study: catching a real solver change between builds

- The cross-version drift suite ran identical inputs on build 5012026 (26.101) and build 7012026 (26.120).
- Every synthetic metric: zero delta. One local body case: pitching moment moved 0.8 percent.
- Triage: reruns bit identical on both builds (not noise), identical input file hash (not setup), single boundary (localized). Verdict: a deterministic solver-side change between builds.

Why this matters to you

This is the exact failure mode the library exists for: two builds, same script, quietly different answers. The version-aware design turns it from a mystery into a measured, documented fact with a report you can cite.

Section outline I

1 Motivation and Installation

2 The Mental Model

3 Architecture

4 First Contact in Ten Minutes

5 The Curated Helpers

6 Recipes: Step by Step per Simulation Type

7 Reading Results

8 Versions and Evidence in Practice

9 Real Solver Pitfalls, Evidence Backed

10 Where the Confidence Comes From

11 Extending and Reference Sheets

Extending the library

- **Add a command:** draft the YAML entry with its evidence citation (the manual page, or a committed probe report), typed arguments, layout, and phase; Tier 1 validates the schema; a probe spec promotes it later. The repository ships a guided workflow for this.
- **Onboard a new FlightStream version:** register it in the ordered list, backfill documented statuses from its manual with citations, then probe on a licensed machine.
- **Contribute a recipe:** recipes are plain functions; ship them with your project or propose the pattern for the docs.

The rules that keep it trustworthy

No manual text is ever reproduced (paraphrase plus citation only). No status is hand edited. Supported versions are never dropped. Every example is synthetic and reproducible from code.

Cheat sheet: phases and layouts

Phases, in order

1. geometry (import, mesh operations)
2. setup (fluid, free stream, actuators, motions)
3. init (initialize, solver settings)
4. exec (START_SOLVER)
5. analysis (loads selection)
6. export (spreadsheets, fields, logs)

control commands (STOP, PRINT, log export) may appear anywhere.

Layout grammars

- bare: name alone
- inline: arguments on the command line
- param lines: KEY VALUE lines ended by a blank line
- payload lines: a count, then that many data lines
- keyword block: KEY VALUE lines until a terminator

The builder renders all five; you never format them by hand.

Cheat sheet: the five command-line tools I

```
pyfs-qa probe             --fs-version ... --fs-exe ...   # Tier 2 probes
pyfs-qa apply-compat      REPORT.yaml                     # promote statuses
pyfs-qa physics           --workspace DIR [--resume]      # the workspace's physics matrix, reduced and reported
pyfs-qa drift             --workspace DIR --fs-versions 26.101,26.120 --fs-exe 26.101=A.exe --fs-exe 26.120=B.exe
pyfs-qa cases                                             # print the case registry
pyfs-qa update-reference --case ... --reason "..."        # guarded write
pyfs-qa cost              --workspace DIR                 # what a run cost
pyfs-workspace init       DIR                             # scaffold a workspace
pyfs-workspace archive    DIR SIM_ID                      # zip one recorded simulation
pyfs-workspace migrate-geometries DIR                     # flat library to one folder per geometry
pyfs-matrix upgrade       matrix.fs --in-place            # older layout to current
pyfs-matrix convert       matrix.fs -o campaign.toml      # run matrix to campaign
pyfs-matrix inspect-setups matrix.fs --workspace DIR # resolved setup and provenance
pyfs-matrix plan          matrix.fs                       # pre-flight, no solver
pyfs-matrix run           matrix.fs --fs-exe ...          # RUNS the solver
pyfs-matrix inventory     30_WB.fsm                       # boundary order, read from the mesh
pyfs-matrix degenerate    blade.obj --root-offset 0.05 # the thin blade of a blade mesh
pyfs-matrix audit-mesh    mesh.obj [--against SRC.obj]  # judge a panel mesh, write its audit
pyfs-matrix refine        mesh.obj 2 [--out DIR]          # refine or coarsen into a new level folder
pyfs-matrix collect       --workspace DIR [--watch]       # waits for a submitted job's outputs
pyfs-matrix post          matrix.fs --workspace DIR       # rebuilds the products, no solver
pyfs-matrix rename        --workspace DIR [--dry-run]  # 0.20.x names to the 0.21.0 point names
pyfs-matrix space-in-use --workspace DIR                  # sizes: folders, sims, extensions
pyfs-matrix free-space    m001 --workspace DIR [--apply]  # a storage recipe
```

Cheat sheet: the five command-line tools II

```
pyfs-matrix delete-sims  4001,2009 --workspace DIR [--apply]  # sims, products, records
pyfs-matrix sync         runs --workspace DIR [--apply]  # from sync-workspaces.toml
pyfs-matrix restore      runs --workspace DIR [--apply]  # a records file, back from archive/
pyfs-matrix rebuild      --workspace DIR [--apply]    # run records again, from the sims/ folders
pyfs-matrix mark-failed  --sims 2006 [--apply]        # a run found wrong, FAILED_MARKED
pyfs-matrix mark-converged --sims 2006 --reason TEXT [--apply]  # your verdict, CONVERGED
pyfs-matrix status       --workspace DIR              # per polar: points, states, freshness
pyfs-matrix show         9101_1 --workspace DIR       # one point's card, outcome first
pyfs-matrix log          --workspace DIR [--problems] # recorded activity, post and storage
pyfs-matrix trace        post/demo/polars/polar.csv   # a product back to its runs
pyfs-matrix history      9101 --workspace DIR         # every recorded copy of a run
pyfs-matrix diff         RUN_A RUN_B --workspace DIR  # two runs, field by field
pyfs-fsi init-dummy      --node-count N [--dir DIR]   # dummy config for a dry run
pyfs-fsi step            [--dir DIR]                  # one coupling step
pyfs-manual sweep        --editions MANIFEST [--by-section]  # every registered edition
pyfs-manual surface      --editions MANIFEST [--names]     # what changed between builds
pyfs-manual coverage     --manual ... --source SRC-741 --chapter-pages 275-369
pyfs-manual citations    --editions MANIFEST   # exits 1 if one does not hold
pyfs-manual register  --editions MANIFEST --fs-version 26.123  # carries a build forward
pyfs-manual draft     --editions MANIFEST --fs-version 26.123  # propose rows for review
```

- The `pyfs-qa` tools (and `pyfs-fsi`) require a licensed machine and write dated report pairs (markdown plus YAML) under `reports/`; `pyfs-qa cases` runs nothing.

Cheat sheet: the five command-line tools III

- `pyfs-workspace`, and `pyfs-matrix convert`, `plan`, `collect` and `post`, run anywhere: they scaffold the workspace tree, turn a run matrix into a campaign, pre-flight one without a solver, wait for a submitted job's outputs and rebuild the products. `collect` starts no solver and is not a pre-flight: it watches the WORKSPACE rather than the queue, which is why it needs neither a licence nor a scheduler. `pyfs-matrix run` STARTS THE SOLVER and needs a licensed machine like the `pyfs-qa` tools. There is no separate submit command: on Linux with a cluster profile in `inputs/hpc/`, `run` hands the job to the scheduler and returns, and the record reads SUBMITTED until `collect` completes it.
- As a user you consume the licensed results through the matrix; you only run the licensed tools when contributing evidence.

Closing: the contract between you and the library

The library promises

- It will never emit a command it cannot back with evidence for your declared version.
- Every refusal explains itself with the cause and a citation.
- Every run under the campaign loop lands in exactly one recorded status with reproducible provenance.

You promise

- The FlightStream version is declared, not assumed.
- Executable and geometry paths are explicit and absolute.
- When you meet undocumented solver behavior, it becomes a probe spec or an issue, so the next user inherits evidence instead of folklore.